

CHAPTER 4: METHODOLOGY AND IMPLEMENTATION

4.1 Design Methodology

4.2 Processor Architecture

4.2.1 Arithmetic Logic Unit (ALU)

4.2.2 Register File

4.2.3 Program Counter (PC)

4.2.4 Control Unit

4.2.5 Instruction Decoder

4.2.6 Static Random Access Memory (SRAM)

4.2.7 System Bus Architecture

4.3 CPU Implementation Using Logisim Evolution

4.3.1 Design of Individual Modules

4.3.2 Integration of CPU Modules

4.3.3 Functional Verification

4.4 FPGA Implementation Using Verilog HDL

4.4.1 Top-Level CPU Module

4.4.2 Clock Divider Module

4.4.3 Seven-Segment Display Controller

4.4.4 Pin Configuration of Basys 3 FPGA

4.5 Deployment on Basys 3 FPGA

4.5.1 Vivado Synthesis

4.5.2 Bitstream Generation

4.5.3 Hardware Programming and Verification

CHAPTER 5: RESULTS AND DISCUSSION

5.1 Logisim Simulation Results

5.1.1 Fetch Cycle Verification

5.1.2 Immediate Value Copy Operation

5.2 FPGA Hardware Verification

- 5.2.1 CPU Output on Seven-Segment Display
- 5.2.2 Register and Memory Verification
- 5.2.3 Hardware Execution Results
- 5.3 FPGA Resource Utilization
- 5.4 Timing Analysis
- 5.5 Comparison Between Simulation and FPGA Results
- 5.6 Performance Evaluation
- 5.7 Advantages and Limitations
- 5.8 Chapter Summary

CHAPTER 4

METHODOLOGY AND IMPLEMENTATION

4.1 Design Methodology

The ANVIKSHA 8-Bit CPU was developed using a systematic design methodology that combined digital circuit simulation with FPGA-based hardware implementation. The processor architecture was initially designed and verified in Logisim Evolution, where individual modules were developed and integrated to implement the complete fetch–decode–execute cycle. After successful simulation, the design was translated into Verilog HDL and implemented on the Basys 3 FPGA using the Xilinx Vivado Design Suite. This methodology ensured functional correctness throughout the development process while providing a smooth transition from simulation to hardware implementation.

4.2 Processor Architecture

The ANVIKSHA processor is based on an 8-bit architecture designed to demonstrate the fundamental concepts of computer organization and processor design. The architecture consists of functional modules such as the Arithmetic Logic Unit (ALU), Register File, Program Counter, Control Unit, Instruction Decoder, and SRAM, all interconnected through a common system bus. The processor follows the Fetch–Decode–Execute cycle, enabling efficient

instruction processing and data manipulation while maintaining synchronization among all modules.

4.2.1 Arithmetic Logic Unit (ALU)

The Arithmetic Logic Unit (ALU) is the computational core of the processor and is responsible for executing arithmetic and logical operations specified by the instruction set. It performs operations such as addition, subtraction, and basic logical functions using operands obtained from the register file. The output generated by the ALU is stored back in the appropriate register or memory location under the control of the Control Unit.

4.2.2 Register File

The Register File provides temporary high-speed storage for operands, intermediate results, and processed data during instruction execution. Each register stores 8-bit data and enables rapid access by the ALU and Control Unit. The register file supports efficient data transfer between different processor modules and plays a significant role in reducing memory access during computation.

4.2.3 Program Counter (PC)

The Program Counter (PC) is a sequential register that stores the address of the next instruction to be executed. During each clock cycle, the PC supplies the instruction address to memory and automatically updates to the subsequent address unless modified by branch or jump instructions. This mechanism ensures continuous and orderly execution of the program.

4.2.4 Control Unit

The Control Unit coordinates the operation of all processor modules by generating the required control signals based on the decoded instruction. It supervises data movement between registers, memory, and the ALU while maintaining the correct sequence of instruction execution. The Control Unit ensures proper synchronization of all hardware components during the fetch, decode, and execute stages.

4.2.5 Instruction Decoder

The Instruction Decoder interprets the binary instruction fetched from memory and determines the corresponding operation to be performed. Based on the instruction opcode, it activates the appropriate control signals required for

arithmetic operations, data transfer, memory access, or program control. This module acts as the interface between the instruction memory and the processor control logic.

4.2.6 Static Random Access Memory (SRAM)

Static Random Access Memory (SRAM) is used to store both program instructions and data required for processor execution. It provides fast read and write operations, enabling efficient instruction fetching and data access. During execution, the processor continuously communicates with SRAM to retrieve instructions and store computation results.

4.2.7 System Bus Architecture

The System Bus Architecture provides the communication pathway between the processor modules. It consists of data, address, and control buses that facilitate the transfer of information between the ALU, registers, memory, and Control Unit. The bus architecture ensures synchronized communication and efficient execution of processor operations.

4.3 CPU Implementation Using Logisim Evolution

The complete processor architecture was implemented using Logisim Evolution, which served as the primary platform for designing and simulating digital circuits. Each module was individually developed and later integrated to construct the complete CPU. Logisim provided an effective environment for validating processor functionality before hardware implementation.

4.3.1 Design of Individual Modules

Each processor component, including the ALU, Register File, Program Counter, Control Unit, Instruction Decoder, and SRAM, was designed as an independent module. Individual testing was performed to verify the functionality of every module before integrating them into the complete processor architecture.

4.3.2 Integration of CPU Modules

After verifying each functional block, all modules were interconnected to form the complete processor. Proper synchronization between the data path and control path was ensured to enable correct execution of instructions through the fetch–decode–execute cycle.

4.3.3 Functional Verification

The integrated processor was tested using various instructions to verify arithmetic operations, register updates, memory access, and control flow. Simulation results confirmed that all processor modules functioned correctly and produced the expected outputs.

4.4 FPGA Implementation Using Verilog HDL

The verified Logisim design was implemented in Verilog Hardware Description Language (HDL) for deployment on the Basys 3 FPGA. Each processor module was coded independently and integrated into a top-level design. The Verilog implementation enabled synthesis, implementation, and real-time hardware execution using Xilinx Vivado.

4.4.1 Top-Level CPU Module

The top-level CPU module integrates all functional blocks of the processor into a single hardware design. It manages communication between internal modules and controls the overall execution of processor instructions on the FPGA platform.

4.4.2 Clock Divider Module

The Basys 3 FPGA provides a 100 MHz clock, which is too fast for manual observation. A clock divider module was implemented to generate a lower-frequency clock, allowing processor operations to be monitored through the onboard LEDs and seven-segment displays.

4.4.3 Seven-Segment Display Controller

The display controller interfaces the processor outputs with the onboard seven-segment display. It converts internal processor data into display signals and continuously updates the displayed values, enabling real-time observation of execution results.

4.4.4 Pin Configuration of Basys 3 FPGA

The processor inputs and outputs were mapped to the FPGA pins using the Xilinx Design Constraints (XDC) file. Switches were configured as input devices, while LEDs and seven-segment displays were assigned as output devices to facilitate hardware testing and verification.

4.5 Deployment on Basys 3 FPGA

After successful synthesis and implementation, the generated bitstream was downloaded to the Basys 3 FPGA board using the Vivado Hardware Manager. Hardware testing confirmed that the FPGA implementation accurately reproduced the behavior observed during Logisim simulation.

4.5.1 Vivado Synthesis

The Verilog design was synthesized using the Xilinx Vivado Design Suite to convert the HDL description into a gate-level hardware implementation. The synthesis process verified the correctness of the design and prepared it for FPGA implementation.

4.5.2 Bitstream Generation

Following successful implementation, Vivado generated the configuration bitstream required to program the FPGA. This bitstream contains the hardware configuration needed for implementing the processor architecture on the Basys 3 board.

4.5.3 Hardware Programming and Verification

The generated bitstream was programmed into the FPGA using Vivado Hardware Manager. The processor was verified by executing test instructions and observing outputs on the onboard LEDs and seven-segment displays, confirming successful hardware implementation.